Data Structures	Algorithms

Searching

- $O(n)$ * Linear Search
- $O(\log n)$ * Binary Search
- $O(\log n)$ * Recursive binary
- $O(\sqrt{n})$ * Jump Search
- position * Interpolation
- * Exponential Search

Special Algos

- * Kadane's
- * Euclid's
- * Rabin Karp
- * Sieve of Eratosthenes
- * Sliding Window

Sorting

	Comparison	Non Comparison
$O(n^2)$	Bubble	Count
$O(n^2)$	Selection	Radix
$O(n^2)$	Insertion	(Bucket)
$n\log n$	Quick	
$n\log n$	Merge	
$n\log n$	Heap	
	Shell	

Kadane's Algorithm :→ [Maximum Subarray Sum] 6

arr = { 5, −8, 1, 2, −1, 4 }
0 1 2 3 4 5

i	arr[i]	cmax + arr[i]	max(arr[i], cmax + arr[i])	gmax m(c, g)
1	−8	5+(−8) = −3	m(−8, −3) = −3	5
2	1	−3+1 = −2	(1, −2)	5
3	2	1+2 = 3	(2, 3)	5
4	−1	3+(−1) = 2	(2, −1)	5
5	4	4+2 = 6	(6, 4)	(6, 5) 6

```
int cmax = arr[0];
int gmax = arr[0];
for (int i=1; i<n; i++)
{
    cmax = max(arr[i], cmax + arr[i])
    gmax = max(cmax, gmax)
}
return gmax;
```

Binary Search :→ Prerequisite (Sorted Array)

2,	3,	5,	6,	9,	15
0	1	2	3	4	5
s	e		s		e

```
s = 0
e = 1
while (s<e)
if (arr[mid] == key)
    return mid;
else if (arr[mid] > key)
    e = mid-1;   → left side
else
    s = mid+1;   → right side
return -1;
```

k = 3
k = 9

mid = (s+e)/2 = (0+5)/2 = 2

arr[4] = 9

3+5 / 2 = 4

(0+1)/2 = 1/2 = 0

s, e

$m = \frac{s+e}{2}$

(OOB)

$(\lfloor m \times 1 \rfloor) = \lfloor m \rfloor$ (−ve)

$m = s + \frac{e-s}{2}$

2^k

$k = 0, 1, 2, 3$

$k = \log_2 N$

$k = \log N$

Big O(log N)

$\frac{N}{2^0}$ N

$\frac{N}{2^1}$ $\frac{N}{2}$ $\frac{N}{2}$

$\frac{N}{2^2}$ $\frac{N}{4}$ $\frac{N}{4}$ $\frac{N}{4}$ $\frac{N}{4}$

$\frac{N}{2^3}$ $\frac{N}{8}$ $\frac{N}{8}$ $\frac{N}{8}$ $\frac{N}{8}$ $\frac{N}{8}$ $\frac{N}{8}$ $\frac{N}{8}$ $\frac{N}{8}$

Every iteration array is broken down in this form → $\frac{N}{2^k}$ where k = 0, 1, 2

∴ $k = \log_2 N$ → O(log N)

7	6	4	3

n = 4

$\boxed{n = 4}$

(n−1) iterations

* Every iteration the largest element will traverse to the end of the array till that part.

* Repetitive Swapping of adjacent elements

Round 1
Pass 1
Iteration 1

0			n−1
7	6	4	3
6	7	4	3
6	4	7	3
6	4	3	7

(n−2) (n−1) 3

j → 0 to < n−1

Pass 2

6	4	3	7
4	6	3	7
4	3	6	7

j < n−1 (n−2)

Pass 3

4	3	6	7
3	4	6	7

for (0 − n)
for (0 − n)
$O(n^2)$